Phase 1 of 8: Core language foundations · lesson 1 of 20

C++ Basics and Pointers

Start here if C++ is new or rusty: types, functions, where variables live, and how pointers and references work.

How a C++ program is built



Compile error: a problem inside one file. Linker error: declared and called but never defined.

Three stages. Know which one an error comes from.

C++ is **compiled** to machine code before it runs: fast, but it trusts you not to make memory mistakes.

main.cpp

```cpp
#include <iostream>          // paste in the declarations for std::cout

int main() {                 // execution starts here
    std::cout << "Hello, C++\n";
    return 0;                // 0 means success
}
```

1. **Preprocess**: `#include` literally pastes header files in; `#define` does text substitution.
2. **Compile**: each `.cpp` file becomes an *object file* on its own.
3. **Link**: the linker joins object files and libraries into one executable and resolves calls between files.

> HABIT TO BUILD NOW Always compile with `-Wall -Wextra`. The compiler warns about many of the bugs this lesson describes (uninitialized variables, unused values, signed/unsigned comparisons).

KEY POINT "Compile error vs linker error?" A compile error is a syntax or type problem inside one file. A linker error ("undefined reference") means a function was declared and called but never defined anywhere you linked.

Variables and types

Every variable has a **type** that fixes how much memory it uses and what you may do with it.

TYPE	HOLDS	TYPICAL SIZE
`bool`	`true` / `false`	1 byte
`char`	a character / small integer	1 byte (always, by definition)
`int`	whole number	4 bytes
`long long`	bigger whole number	8 bytes
`float` / `double`	decimal number	4 / 8 bytes

Only `char` has a fixed size. When size matters use `<cstdint>`: `std::int32_t`, `std::uint64_t`.

Initialization: always give a variable a value

```
// BAD: indeterminate value, reading it is undefined behavior
int a;
int b = 5;      // copy initialization
int c(5);       // direct initialization
int d{5};       // brace (uniform) initialization: preferred
int e{};        // empty braces: zero-initialized, e == 0
int f{3.7};     // ERROR: braces forbid narrowing double → int
auto g = 5;     // g is int: the compiler deduces the type
const int h = 9;// cannot be changed after initialization
```

UNDEFINED BEHAVIOR An uninitialized local holds garbage. Reading it is **UB**: junk, a crash, or "works" until it does not.

The signed/unsigned trap

`unsigned` wraps instead of going negative, and a mixed comparison converts the signed side to unsigned.

Operators and control flow

```
7 / 2      // 3     integer division truncates
7 % 2      // 1     remainder
// 3.5 one double operand makes the whole thing floating point
7 / 2.0

// && and || short-circuit: right side is skipped if not needed
if (x > 0 && y > 0) { }
for (int i = 0; i < 3; ++i) { }
for (int v : vec) { }     // range-based for: visits every element
while (cond) { }
// forget `break` and it falls through
switch (n) { case 1: /* ... */ break; default: break; }
```

> CLASSIC BUG `if (x = 5)` *assigns* 5 to x and is always true. You meant `==`. The compiler warns with `-Wall`.

Functions and passing arguments

Declare before use; the header holds declarations, the .cpp the definitions.

```
int add(int a, int b);                  // declaration
int add(int a, int b) { return a + b; }// definition
// overload: same name, different parameter types
double add(double a, double b);
void greet(std::string who = "world"); // default argument
```

Three ways to pass a parameter

```
void byValue(int n)         { n++; }          // works on a COPY
void byRef(int& n)          { n++; }          // works on the ORIGINAL
// original, read-only, no copy
void byConstRef(const std::string& s) { }
```

YOU WANT TO...	USE
read a small value (int, double, char, pointer)	by value: `int n`
read a large object without copying it	by const reference: `const std::string& s`
modify the caller's variable	by reference: `int& n`
accept "no object" as a valid argument	by pointer: `Widget* w`, and test it for `nullptr`
pass a run of elements	`std::span<int>` (C++20) or `const std::vector<int>&`, not a pointer plus a count
get a result back	return it by value; big objects such as `std::vector` are moved, not copied, so no out-parameter is needed

---

Pointers from scratch

A **pointer** is a variable whose value is an **address**.

SYNTAX	MEANING
`int* p`	declare `p` as "pointer to int"
`&x`	address-of: the address of `x`
`*p`	dereference: the thing `p` points to
`nullptr`	the pointer that points to nothing

Pointers and arrays

An array name **decays** to a pointer to its first element; `p + 1` moves one *element*.

```
int a[3] = {10, 20, 30};
int* p = a;            // same as &a[0]
std::cout << *p;       // 10
std::cout << *(p + 1); // 20   (p + 1 advances by sizeof(int) bytes)
std::cout << p[2];     // 30   p[i] is exactly *(p + i)
```

CAREFUL Never dereference `nullptr` and never read past the end of an array. Both are undefined behavior. C++ will not stop you.

WHAT A POINTER DOES NOT KNOW Only an address and a type: not **how many** elements, nor **who** frees them. Root of most pointer bugs.

KEY POINT "What is the difference between `NULL` and `nullptr`?" `NULL` is a macro for a null pointer constant, usually plain `0` (an integer), so `f(NULL)` can call `f(int)` instead of `f(char*)`. `nullptr` has its own type and only converts to pointers.

References

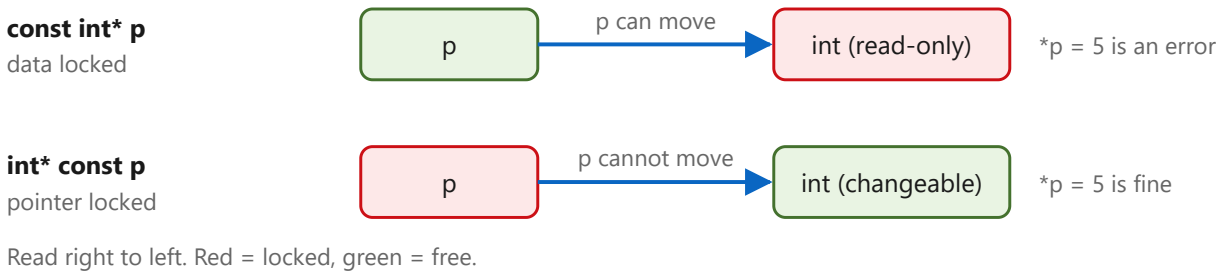
A reference is an **alias**: another name for an existing variable, not a copy.

```
int x = 1;
int& r = x;     // r is another name for x
r = 5;          // x is now 5
```

	POINTER `INT*`	REFERENCE `INT&`
Can be null	yes	no (must refer to something real)
Must be initialized	no (but should be)	yes, at declaration
Can be re-pointed	yes	no, bound for life
Syntax to use	`*p`, `p->member`	just use the name
Use for	optional things, arrays, ownership, C APIs	function parameters, aliases

---

`const` and pointers



const before the star protects the data; after the star it protects the pointer.

`const` = "I will not modify this". With pointers, either the data or the pointer can be const.

Read right to left.

```
int x = 1, y = 2;

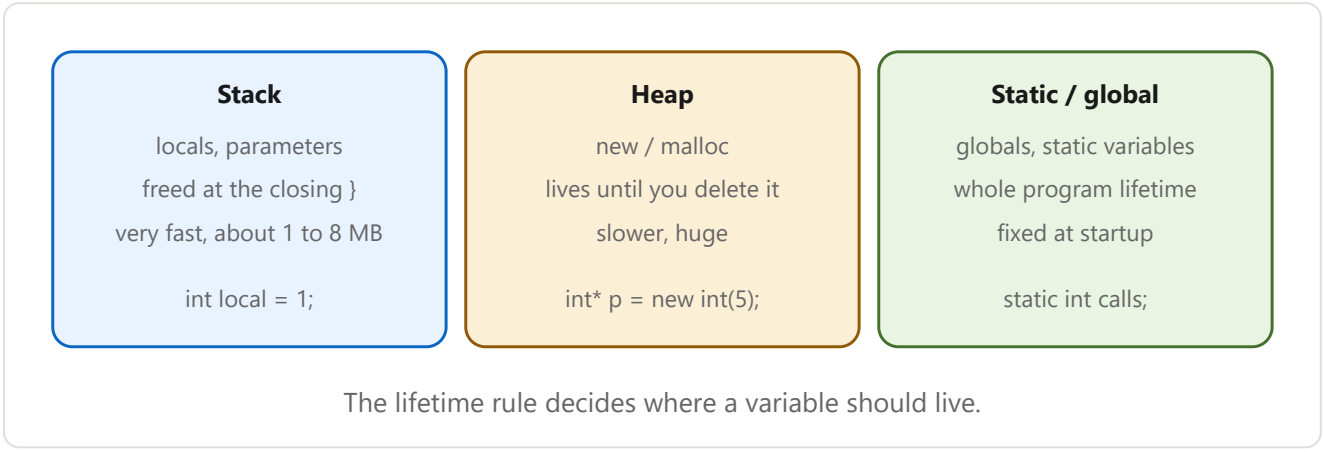
const int* p1 = &x;          // "p1 is a pointer to const int"
// *p1 = 5;                  // ERROR: cannot change the int through p1
p1 = &y;                     // OK: the pointer itself may move

int* const p2 = &x;          // "p2 is a const pointer to int"
*p2 = 5;                     // OK: the int may change
// p2 = &y;                  // ERROR: the pointer may not move

const int* const p3 = &x;    // neither can change
```

Make everything `const` that can be.

Where variables live: stack, heap, static



The lifetime rule decides where a variable should live.

REGION	WHAT GOES THERE	LIFETIME	SPEED / SIZE
Stack	local variables, function parameters	automatic: freed when the scope `{ }` ends	very fast, small (about 1 to 8 MB)
Heap (free store)	objects created with `new` / `malloc`	manual: lives until you `delete` it	slower, huge
Static / global	globals, `static` variables	whole program	fixed at startup

```
// static storage: exists for the whole program
int counter = 0;

void f() {
    // stack: created on entry, destroyed on exit
    int local = 1;
    // static storage, but only visible inside f
    static int calls = 0;
    ++calls;
    // the int lives on the HEAP; p itself is on the stack
    int* p = new int(5);
    delete p;
}
```

MENTAL MODEL Stack: a pile of plates, popped automatically. Heap: a storage unit you rent until you hand the key back.

Dynamic memory: `new` and `delete`

`new` allocates on the heap **and** constructs; you must `delete` it or it **leaks**.

```
// single object: new / delete
int*  one = new int(42);     delete one;
// array: new[] / delete[] (must match!)
int*  arr = new int[100];    delete[] arr;
```

BUG	WHAT HAPPENED
Memory leak	you lost the pointer (or forgot `delete`) so the memory is never freed
Dangling pointer	you used a pointer after the memory was freed
Double delete	you freed the same memory twice: undefined behavior, often a crash
Mismatch	`new[]` freed with `delete`, or the reverse

MODERN C++ Avoid raw `new` / `delete`: use `std::vector` and `std::unique_ptr` (Smart Pointers).

KEY POINT "What does `new` do beyond `malloc`?" It calls the constructor, throws `std::bad_alloc` on failure (instead of returning null), and returns a correctly typed pointer. `delete` calls the destructor and then frees the memory.

Arrays, C-strings and `std::string`

```
// fixed size, remaining elements are 0, NO bounds checking
int a[5] = {1, 2, 3};
a[10] = 7;                  // compiles, undefined behavior at runtime

// this is really `int* arr`: the size is lost!
void f(int arr[]) {
    sizeof(arr);            // size of a POINTER (8), not of the array
}

char s[] = "hi";            // 3 chars: 'h', 'i', and a terminating '\0'
```

Prefer:

INSTEAD OF	USE	WHY
`int a[N]`	`std::array<int, N>`	knows its size, no decay, same speed
`new int[n]`	`std::vector<int>`	grows, frees itself, `.at(i)` bounds-checks
`char*` strings	`std::string`	owns its memory, safe copy, `+`, `.size()`, `.find()`
pointer plus a count	`std::span<int>` (C++20)	a non-owning view that remembers its size; accepts an array or a vector without copying

```
std::string s = "hello";
s += " world";                          // grows automatically
std::cout << s.size() << s.substr(0, 5);   // 11 hello
std::vector<int> v = {1, 2, 3};
v.push_back(4);                         // size is now 4
```

The classic memory bugs

Most "mysterious" crashes are one of these.

BUG	EXAMPLE
Uninitialized read	`int n; use(n);`
Out-of-bounds	`int a[3]; a[3] = 1;` (valid indices are 0 to 2)
Null dereference	`int* p = nullptr; *p = 1;`
Use after free	`delete p; *p = 1;`
Signed overflow	`INT_MAX + 1` (UB for signed; wraps for unsigned)

Returning a reference to a local

```
int& bad() {
    int local = 42;
    return local;          // BUG: local is destroyed when bad() returns
}                          // the caller gets a dangling reference

int good() {
    int local = 42;
    return local;          // fine: returns a copy of the value
}
```

TOOLING Test with `-fsanitize=address,undefined`: it reports leaks, out-of-bounds, use-after-free and overflow with the exact line.

Tooling: warnings, sanitizers, static analysis

C++ trusts you, so let tools check what the language will not. Use them in layers, cheapest first.

LAYER	WHAT IT CATCHES	HOW
Compiler warnings	many beginner mistakes at once: uninitialized variables, signed/unsigned comparisons, missing returns	`-Wall -Wextra`, and `-Werror` in CI
Sanitizers	what only shows at run time: out-of-bounds, use after free, leaks, undefined behavior, data races	`-fsanitize=address,undefined` (and `-fsanitize=thread` for threads)
Static analyzer	rule violations, null dereferences, leaks and dead code found by reading the source, without running it	`clang-tidy`, `cppcheck`
Formatter	style, so reviews are about logic	`clang-format`

A CLEAN REPORT IS NOT A PROOF Analyzers give false positives and miss things, and a sanitizer only checks the paths your test actually ran. They reduce risk; they do not replace design, review and tests.

CODING STANDARDS Safety-critical projects (cars, medical devices) add written rules such as MISRA C++ or the AUTOSAR C++14 guidelines, checked automatically. The common themes are the ones in this course: initialize everything, no C-style casts, no naked `new` / `delete`, no silent narrowing, and no `using namespace` in headers.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
How a C++ program is built	C++ Primer	2.6 Defining our own data structures	101 to 109
	C++ Primer	3.1 Namespace using declarations	111 to 112
	C++ Primer	18.2 Namespaces	814 to 830
Variables and types	Programming: Principles and Practice	2.5 Assignment and initialization	49 to 52
	Programming: Principles and Practice	2.8 to 2.10 Type safety, conversions, type deduction with auto	56 to 60
	Effective Modern C++	Item 7, Distinguish () and {} when creating objects	67 to 76
Operators and control flow	C++ Primer	4.2 to 4.3 Arithmetic, logical and relational operators	168 to 172
	C++ Primer	4.4 to 4.5 Assignment, increment and decrement	173 to 178
	C++ Primer	5.3 to 5.5 Conditional, iterative and jump statements	203 to 221
Functions and passing arguments	C++ Primer	6.1 Function basics	231 to 236
	C++ Primer	6.2 Argument passing	237 to 250
	C++ Primer	6.3 Return types and the return statement	251 to 258
	C++ Primer	6.4 Overloaded functions	259 to 264
	C++ Primer	6.6 Function matching	271 to 275
	Programming: Principles and Practice	3.5 to 3.6 Functions; vector introduced	81 to 89
	Programming: Principles and Practice	7.2 to 7.3 Declarations and definitions; scope	194 to 202
	Programming: Principles and Practice	7.4 Function call and return (pass by value, by reference, by const reference)	203 to 218
Pointers from scratch	C++ Primer	2.3 Compound types	79 to 87
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463
References	C++ Primer	2.3 Compound types	79 to 87
	Programming: Principles and Practice	16.1 to 16.2 Arrays, pointers and references	477 to 483
const and pointers	C++ Primer	2.4 const qualifier	88 to 95

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Where variables live: stack, heap, static	Programming: Principles and Practice	7.2 to 7.3 Declarations and definitions; scope	194 to 202
Dynamic memory: new and delete	C++ Primer	12.2 Dynamic arrays	505 to 512
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463
Arrays, C-strings and std::string	C++ Primer	3.2 Library string type	113 to 124
	C++ Primer	3.5 Arrays	142 to 153
	C++ Primer	3.6 Multidimensional arrays	154 to 161
	Programming: Principles and Practice	16.3 to 16.4 C-style strings; alternatives to pointer use	484 to 487